Software Engineering in the Enterprise

Understanding the roles, responsibilities, and collaborative tools essential for success in real-world software development.

Made with GAMMA

Diverse Roles in Software Development

1

Product Manager

Defines product vision and requirements.

2

Backend Developer

Builds server-side logic and databases.

3

Frontend Developer

Crafts user interfaces and experiences.

4

QA / Tester

Ensures software quality and identifies defects.

5

DevOps / Platform

Manages infrastructure, deployment, and operations.

6

Security / AppSec

Integrates security practices throughout the SDLC.

Made with GAMMA

Responsibility and Ownership

Each role carries distinct responsibilities, fostering accountability and expertise within the team.

- Clear division of tasks prevents overlap and enhances efficiency.
- Ownership ensures a high standard of quality for specific components.
- Understanding these boundaries is crucial for effective collaboration.

Backend Developer Core Responsibilities:

- **Business Logic:** Implementing the core functionalities and rules of the application.
- **Data Integrity:** Ensuring data accuracy, consistency, and security in databases.
- **API Behavior:** Designing and maintaining reliable and efficient application programming interfaces.

Made with GAMMA

CHAPTER B

The Software Development Lifecycle (SDLC)

The SDLC outlines the structured process for building, deploying, and maintaining software effectively.

01

Requirement Gathering

Defining what the software needs to achieve.

02

Design & Planning

Architecting the solution and creating technical specifications.

03

Implementation

Writing the actual code.

04

Code Review

Peer evaluation to ensure quality and catch issues.

05

Testing & Quality Assurance

Verifying functionality and performance against requirements.

06

Deployment

Releasing the software to users.

07

Monitoring & Maintenance

Observing performance, identifying bugs, and applying updates.

Made with GAMMA

Git's Pivotal Role in the SDLC

Git, a distributed version control system, is indispensable for modern software development, particularly during these critical phases:

Implementation

Tracking every code change, enabling rollback and parallel development.

Code Review

Facilitating structured review processes for improved code quality.

Collaboration

Allowing multiple developers to work on the same codebase seamlessly.

Traceability

Providing a complete history of changes, crucial for debugging and auditing.

Made with GAMMA

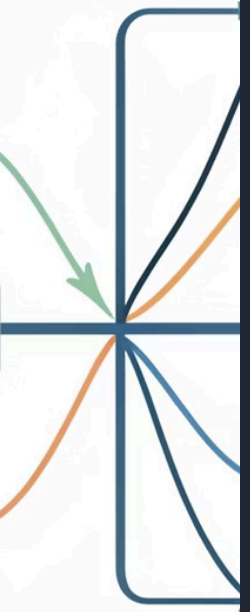
CHAPTER C

Beyond Storage: The True Power of Git

Git is more than just a repository; it's a fundamental tool for team collaboration and project resilience.

- **Track Changes:** Git maintains a detailed history of every modification, showing who made what changes and when.
- **Review Code:** It enables efficient code review workflows, fostering knowledge sharing and bug prevention.
- **Identify Responsibility:** Git's commit history clearly attributes changes, crucial for accountability and debugging.
- **Recover from Failures:** Easily revert to previous stable versions, minimizing downtime and data loss.

Made with GAMMA



CHAPTER D

Our Git Workflow for This Course

To ensure organized and collaborative development, we will adhere to a standardized Git workflow.

Protected `main` Branch

The `main` branch serves as the stable production codebase and is protected from direct pushes.

Feature Branches

All new development, bug fixes, or experimental features are implemented within dedicated feature branches.

Pull Request (PR)

Changes are integrated into `main` only after thorough review and approval via a Pull Request.

Made with GAMMA

Rules of Collaboration

Adhering to these guidelines ensures code quality, team efficiency, and project stability.

1 No Direct Push to `main`

All changes must go through the Pull Request process for review and approval.

2 Every Change Requires Review

A peer review is mandatory for all code modifications before merging.

3 Merging Means Responsibility

The developer merging a PR is accountable for its quality and potential impact.

Made with GAMMA

Crafting Effective Commit Messages

Clear and concise commit messages are vital for understanding code history and project evolution.

feat: add new feature

Introduces a new capability or significant enhancement.

fix: resolve bug

Addresses a bug or corrects an error in functionality.

chore: update configs

Routine tasks that don't affect code logic or user-facing features (e.g., build process, dependency updates).

docs: update README

Changes related to documentation (e.g., README, inline comments).

Adopt a consistent convention like Conventional Commits to maintain a clean and informative Git log.

Made with GAMMA

Key Takeaways for Future Security Professionals

- **Collaboration is Key:** Software development is a team sport; understand your role and others'.
- **Security is Everyone's Job:** Integrate security thinking into every stage of the SDLC.
- **Master Git:** It's not just version control; it's a powerful tool for quality, collaboration, and accountability.
- **Attention to Detail:** From commit messages to code reviews, precision matters.

Your understanding of these principles as future Information Security professionals will be invaluable in securing the software you build and audit.

Good luck!

Made with GAMMA